HTML Quality Detection for Muck Rack Article Ingestion Pipeline

By: Rishabh Patil, Deepali Balakrishna Ksheersagar, Kund Nilesh Meghani

NYU Center for Data Science

Context

Introduction

Muck Rack faces a persistent challenge in its article ingestion pipeline: a high volume of downloads result in "silent" failures. Even with a successful 200 HTTP response, the returned HTML is often unusable, which current detection methods miss. Manually reviewing even a fraction of this content is infeasible given the sheer scale of Muck Rack's data.

This presents an excellent use case for Machine Learning. Our goal was to build an ML model that automatically assesses the content and other features from the HTMLs. By ingesting the raw HTML and extracted features, our model predicts two classes, enabling efficient filtering and quality control at scale.(105 words)

Normal vs Broken HTML

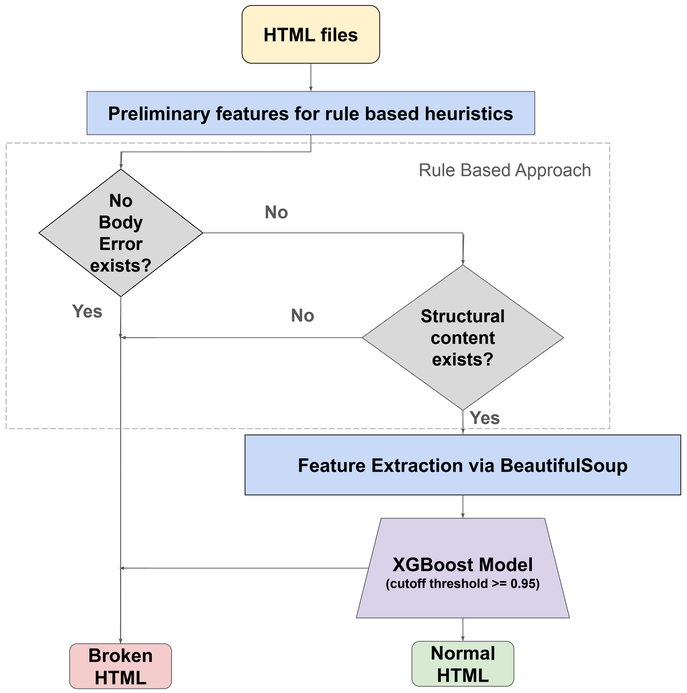
We consider a HTML as **"Broken"** if there is no parsed or any useful information from it. For example:

- The HTML is completely empty.
- The HTML only includes some information related cookies or privacy policy.}
- The HTML is not decoded correctly.
- The HTML only contains Javascript to dynamically load content or load content in via a Shadow DOM.

Classification Pipeline

Components

Add Flowchart Here (TODO)



Rule-Based Heuristics

Structural content detection (article-like signals)

Flag as **qualified** if the page contains strong content evidence such as:

- **Long meta description** (e.g., name=description or property=og:description)
- Substantial text inside **<article> or <main>**
- **Content identifiers like** div/section/p with class/id and matching patterns like *content/article/story/news* **and** meaningful text
- **High paragraph density** (enough paragraphs + total paragraph length)
- **Large text blocks inside <td> (tables)**

If these signals are absent, flagged as **Broken**.

No-body / empty-page detection

- Compute word counts for: **total visible text**, text inside **<body>**, and text inside **<head>**
- If **body text is below a threshold** and overall visible text is also low → classify as **Broken**
- **Captures empty shells, failed downloads, or pages with no extractable body content**

XGBOOST Modelling

Structural features

- **num_tags_total** — Total number of HTML tags in the document; reflects structural complexity.
- **missing_head_flag** — Indicates whether the <head> tag is missing.
- **script_tag_ratio** — Count of <script> tags to count of total tags
- **text_html_ratio** – Ratio of visible text size to raw HTML size

Content features

- **text_length** — Total number of characters in the visible text.
- **max_paragraph_length** – Length of the longest text paragraph
- **num_sentences** – Count of sufficiently long, sentence-like text spans
- **avg_sentence_length** – Average character length of detected sentences
- **special_chars** — Count of punctuation characters.
- **num_digits** — Count of numeric characters.
- **empty_title_flag** — Indicates whether the page title is missing or empty.
- **digit_ratio** — Number of digits to total characters extracted from the page

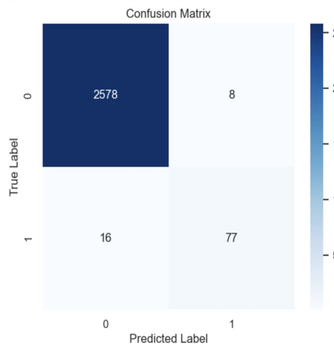
Dataset and enrichment

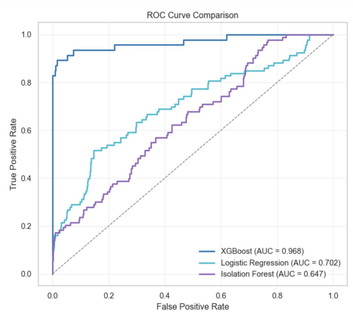
The initial labeled pool was highly imbalanced,with **only ≈ 38 broken HTML pages versus ≈ 2500 normal pages.** This limited our ability to train and evaluate reliable classifiers, especially for the rare "broken" failure modes that matter most in production

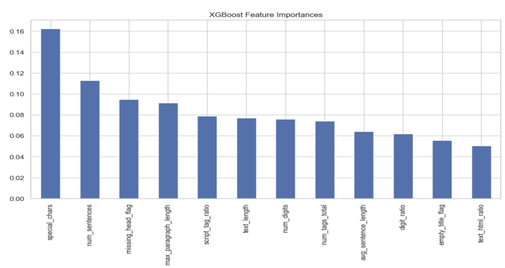
Data Enrichment

- We Built a **GPT-assisted offline labeling pipeline** to expand a small, imbalanced seed set (dataset expansion only).
- For each HTML we extract visible text + compute keyword/regex cues + include short evidence snippets; query **gpt-4.1-mini** (temp=0) with **strict JSON** output.
- Added **few-shot calibration examples** and **deterministic settings** to keep labels consistent and reduce noisy outputs.
- This Expanded the dataset to **~17,500 labeled samples**

Results & Discussion







Function	Mean Calls	Mean Time (s)	Cumulative Time (s)
process_one_html_full	1.0	0.000062	0.032132
extract_all_features	1.0	0.000198	0.025701
classify_n_files_for_profile	0.01	0.000316	0.032705
feed	1.0	0.002404	0.017587
handle_starttag	752.0	0.000866	0.006619
endData	2313.0	0.001269	0.004028

Table 1: Profiling results of major functions in the HTML quality detection pipeline.

```
Best Threshold: 0.9539078156312625
Best F1: 0.9005847953216374
Precision: 0.9871794871794872
Recall: 0.8279569892473119
```

The **XGBoost classifier** outperformed both Isolation Forest and Logistic Regression on given dataset and the selected feature set. At operating threshold of **0.95**, chosen to maximize validation F1-score while **minimizing false positives,** the model achieved a **precision of 0.98, recall of 0.82, and an F1-score of 0.90**.

Conclusion

This poster presents a practical, scalable solution for detecting broken or unqualified HTML in large-scale web ingestion pipelines, where a successful HTTP response does not guarantee extractable content.

Overall, the proposed approach offers a reliable and deployment-friendly mechanism to reduce downstream extraction failures and improve data quality at scale.

Future work includes monitoring for emerging failure patterns, periodic retraining to adapt to shifting website templates, and threshold tuning based on downstream cost trade-offs (e.g prioritizing recall when missed broken pages are more expensive than occasional false alarms)